

Signals and Systems Sessional (ICE-2204)

Laboratory Experiments – Objective and Theory

Experiment 1: Elementary Discrete-Time Signals

Objective

- To understand the mathematical definitions of the unit sample (impulse), unit step, and unit ramp sequences.
- To generate and plot these elementary discrete-time signals using MATLAB/Python.

Theory

Discrete-time signals are functions defined only at integer values of the independent variable n , and are often represented as a sequence of numbers $x(n)$. Certain elementary sequences form the building blocks from which more complicated discrete-time signals can be constructed, and they are also used extensively to represent the characteristics of discrete-time systems.

$$\delta(n) = 1, n = 0; \delta(n) = 0, n \neq 0$$

The unit sample sequence (unit impulse), $\delta(n)$, is zero everywhere except at $n = 0$, where its value is unity. It is analogous to the impulse function in continuous time and is the basic building block used to represent an arbitrary sequence as a weighted sum of shifted impulses.

$$u(n) = 1, n \geq 0; u(n) = 0, n < 0$$

The unit step sequence, $u(n)$, has amplitude 1 for $n \geq 0$ and amplitude 0 for $n < 0$. It can be expressed as the running sum of the unit sample sequence, and conversely, the unit sample can be obtained as the first difference of the unit step.

$$r(n) = n, n \geq 0; r(n) = 0, n < 0$$

The unit ramp sequence, $r(n)$, increases linearly with n for $n \geq 0$ and is zero for $n < 0$. It is related to the unit step sequence through summation, i.e. $r(n)$ is the running sum of $u(n-1)$. These three elementary sequences are generated in MATLAB/Python using logical indexing or built-in array functions (such as zeros, ones, and arange), and are visualised using stem plots, which are the appropriate plotting style for discrete-time signals.

Experiment 2: Basic Sequence Operations – Shifting, Folding and Indexing

Objective

- To determine and plot time-shifted, time-reversed (folded), and index-manipulated versions of a given discrete-time sequence $x(n)$.
- To understand how the index origin (the bold/zero sample) is tracked when a sequence is transformed.

Theory

A discrete-time sequence $x(n)$ is fully described only when both its amplitude values and the corresponding index (time) values are specified, since MATLAB/Python array indices always start from 1

or 0 and do not automatically correspond to the true signal index n . In this experiment, a signal $x(n)$ is defined over a specific range of n , with one particular sample marked as the $n = 0$ reference (the bold value).

Common sequence transformations examined are: (i) time shifting, $x(n - n_0)$, which delays ($n_0 > 0$) or advances ($n_0 < 0$) the sequence along the n -axis without altering its shape; (ii) folding (time reversal), $x(-n)$, which reflects the sequence about the vertical axis $n = 0$; and (iii) combinations of shifting and folding such as $x(-n - n_0)$ or $x(n_0 - n)$. Each operation changes the effective index range of the sequence, so a helper/user-defined function is generally used that keeps track of both the amplitude vector and its associated index vector so the transformed sequence is plotted with the correct n -axis values.

Such operations are fundamental in digital signal processing because convolution, correlation, and system-response calculations are all built from repeated shifting, folding and multiplication of sequences.

Experiment 3: Sequence Addition and Multiplication

Objective

- To determine and plot the sum and product of two discrete-time sequences having different index ranges.
- To implement a general-purpose routine that aligns sequences on a common index axis before performing arithmetic operations.

Theory

Unlike continuous-time signals, arithmetic combination of two discrete-time sequences requires that both sequences first be defined over an identical range of the index n ; samples that fall outside a sequence's original range are treated as zero (zero-padding). If $x_1(n)$ is defined on n_1 and $x_2(n)$ is defined on n_2 , both are re-expressed on the combined range $n = \min(n_1, n_2)$ to $\max(n_1, n_2)$ before addition or multiplication is carried out sample-by-sample.

$$y(n) = x_1(n) + x_2(n) \text{ and } y(n) = x_1(n) \cdot x_2(n)$$

In MATLAB/Python, this is accomplished by writing a user-defined function that accepts each sequence together with its index vector, computes the union of the index ranges, zero-pads both sequences accordingly, and then returns the resulting sequence with its new index vector. The resulting sequence is plotted using a stem plot with the correctly aligned n -axis. This experiment illustrates the practical bookkeeping required whenever signals of unequal length or unequal time support are to be combined, which occurs routinely in filtering, windowing and convolution operations.

Experiment 4: User-Defined Functions for Signal Operations: Addition, Folding and Shifting

Objective

- To design and implement user-defined functions in MATLAB/Python that perform signal addition, folding and shifting on arbitrary discrete-time sequences.
- To plot the results of each operation and verify them against the analytically expected sequence.

Theory

Rather than repeating the same index-alignment logic every time an operation is needed, it is efficient to encapsulate each operation inside its own reusable function. A shifting function accepts a sequence $x(n)$, its index vector n , and a shift amount n_0 , and returns the shifted sequence together with the updated index vector $n + n_0$. A folding function accepts $x(n)$ and n and returns $x(-n)$ together with the negated and re-sorted index vector $-n$. An addition function, as described in the previous experiment, aligns two sequences on a common index range (zero-padding where necessary) and then adds them sample-by-sample.

Building these operations as standalone, reusable functions promotes modular code design and mirrors the way signal-processing toolboxes are structured internally: complex operations such as convolution and correlation can subsequently be built up from these elementary building blocks. Each function is tested on sample sequences and its output is displayed using stem plots so that the correctness of the shifting, folding and addition logic can be verified visually.

Experiment 5: Convolution of Discrete-Time Sequences

Objective

- To understand the mathematical definition of discrete convolution and its role in computing the response of an LTI system.
- To implement the convolution sum in MATLAB/Python, both using a built-in function and a user-defined routine, and to plot the resulting sequence.

Theory

Convolution is the fundamental operation that relates the input of a linear time-invariant (LTI) discrete-time system to its output through the system's impulse response. If $x(n)$ is the input sequence and $h(n)$ is the impulse response of the system, the output $y(n)$ is given by the convolution sum:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k) h(n - k)$$

This expression states that the output at any instant n is a weighted sum of past and present input samples, where the weights are given by the (time-reversed and shifted) impulse response. Conceptually, convolution can be visualised as folding one sequence, shifting it across the other, multiplying overlapping samples, and summing the products for each shift – the classic 'flip, shift, multiply, add' procedure.

In MATLAB, convolution can be computed directly using the built-in `conv` function (in Python, `numpy.convolve` or `scipy.signal.convolve`), or it can be implemented manually with nested loops that carry out the flip-shift-multiply-add steps explicitly, which reinforces understanding of the underlying mathematics. The length of the resulting sequence $y(n)$ equals the sum of the lengths of $x(n)$ and $h(n)$ minus one. Convolution underlies virtually all filtering operations in digital signal processing.

Experiment 6: Generation of Sinusoidal Signals of Different Frequencies

Objective

- To understand the general mathematical form of a discrete-time/continuous-time sinusoidal signal.
- To generate and plot sinusoidal waveforms of different frequencies, amplitudes and phases using MATLAB/Python.

Theory

A sinusoidal signal is one of the most important elementary signals in signal processing, as any periodic signal can be expressed as a sum of sinusoids of different frequencies (Fourier series), and sinusoids are also the eigenfunctions of LTI systems. A general sinusoidal signal is written as:

$$x(t) = A \sin(2\pi f t + \phi) \text{ or in discrete form } x(n) = A \sin(2\pi f n / f_s + \phi)$$

where A is the amplitude, f is the frequency in Hz, f_s is the sampling frequency, and ϕ is the phase angle in radians. In this experiment, a time (or sample-index) vector is first created using an appropriate sampling interval, and sinusoids of different frequencies are generated by evaluating the above expression for different values of f while keeping A and ϕ fixed (or vice-versa). The resulting waveforms are plotted on the same or separate axes to observe how increasing frequency compresses the waveform in time, how amplitude scales the peak values, and how phase shifts the waveform along the time axis. Correct choice of the sampling rate (satisfying the Nyquist criterion, $f_s > 2f$) is essential to avoid aliasing when the signal is generated in discrete form.

Experiment 7: Discrete Fourier Transform (DFT) and Inverse DFT (IDFT)

Objective

- To understand the mathematical definition of the DFT and IDFT and their role in transforming a discrete-time signal between the time and frequency domains.
- To implement the DFT and IDFT in MATLAB/Python, using both direct summation and the built-in FFT-based functions, and to verify perfect reconstruction of the original signal.

Theory

The Discrete Fourier Transform converts a finite-length discrete-time sequence $x(n)$, $n = 0, 1, \dots, N-1$, into a sequence of N complex frequency-domain samples $X(k)$, representing the signal's content at N equally spaced discrete frequencies. It is defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}, k = 0, 1, \dots, N-1$$

The Inverse DFT recovers the original time-domain sequence from its frequency-domain representation:

$$x(n) = (1/N) \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}, n = 0, 1, \dots, N-1$$

The DFT and IDFT can be implemented directly from these summation formulas (which require on the order of N^2 complex multiplications), or far more efficiently through the Fast Fourier Transform (FFT) algorithm, which is available in MATLAB (`fft`, `ifft`) and Python (`numpy.fft.fft`, `numpy.fft.ifft`) and reduces the computational complexity to the order of $N \log N$. In this experiment both approaches are implemented and compared, and the IDFT of the computed DFT is verified to reconstruct the original sequence, confirming the invertibility of the transform pair. The magnitude and phase of $X(k)$ are also plotted to visualise the frequency content of the signal.

Experiment 8: Spectrum of a Given Signal

Objective

- To compute and plot the frequency spectrum (magnitude and phase) of a given discrete-time signal using the DFT/FFT.

- To interpret the spectral plot in terms of the dominant frequency components present in the signal.

Theory

The spectrum of a signal describes how its energy or amplitude is distributed across frequency. For a discrete-time signal $x(n)$ of length N , the spectrum is obtained by computing its N -point DFT, $X(k)$, using the FFT algorithm. The magnitude spectrum, $|X(k)|$, indicates the strength of each frequency component, while the phase spectrum, $\angle X(k)$, indicates the relative phase of each component.

Because the raw DFT index k corresponds to a normalised digital frequency, the frequency axis is generally converted into physical units (Hz) using the relation $f = k f_s / N$, where f_s is the sampling frequency. For real-valued input signals, the spectrum is symmetric about the midpoint (Nyquist frequency), so it is common to plot only the first half of the spectrum, $k = 0$ to $N/2$. In this experiment, the given signal is sampled at an appropriate rate, its N -point FFT is computed, and the resulting magnitude spectrum is plotted against frequency to identify the dominant sinusoidal components contained in the signal, along with their relative amplitudes.

Experiment 9: Removing Noise from an Audio Signal using DFT-Domain Filtering

Objective

- To generate a pure tone audio signal, corrupt it with random noise, and remove the noise using frequency-domain (DFT-based) filtering.
- To understand the practical application of the DFT/IDFT pair in denoising real-world signals.

Theory

Random noise typically spreads its energy across a wide range of frequencies, whereas a pure tone concentrates almost all of its energy at a single frequency. This difference in spectral behaviour is exploited to separate signal from noise in the frequency domain. The procedure followed in this experiment is:

1. A clean audio signal is generated as a pure sine wave of 440 Hz (the musical note A4), sampled at a suitable sampling rate over a chosen duration.
2. Random (e.g. Gaussian white) noise is added to this clean signal to simulate a noisy recording, $y(n) = x(n) + w(n)$.
3. The Discrete Fourier Transform of the noisy signal is computed, converting $y(n)$ into its frequency-domain representation $Y(k)$, which reveals a strong peak at 440 Hz superimposed on a broadband noise floor.
4. A frequency-domain filter is applied by attenuating or zeroing out the DFT coefficients corresponding to frequencies far from 440 Hz (i.e., removing/suppressing the high-frequency noise components while retaining the coefficients near the signal's true frequency).
5. The Inverse DFT is applied to the filtered spectrum to reconstruct a cleaned time-domain signal, which is compared with the original noisy and clean signals, both visually (waveform plots) and audibly.

This experiment demonstrates a fundamental principle of DSP-based noise reduction: transforming a signal into a domain (frequency) where the desired signal and the unwanted noise are more easily

separated, performing the removal there, and transforming back to the original (time) domain.

Experiment 10: Feature Extraction from a PPG Signal

Objective

- To extract clinically relevant features – systolic peaks, diastolic points, and heart rate – from a Photoplethysmogram (PPG) signal.
- To apply basic signal-processing techniques (filtering and peak detection) to a real physiological signal.

Theory

A Photoplethysmogram (PPG) is an optically obtained signal that reflects the volumetric change of blood in the peripheral circulation with each heartbeat. Each cardiac cycle in a PPG waveform typically shows a rapid up-slope corresponding to the systolic peak (the point of maximum blood volume during ventricular contraction), followed by a decline that may include a smaller local feature known as the diastolic point (or dicrotic notch), corresponding to the reflection of the pressure wave during the diastolic phase of the heart.

Since raw PPG signals are often corrupted by baseline drift and high-frequency noise (e.g. from motion artifacts), a pre-processing (filtering) stage, such as a band-pass filter, is usually applied first to isolate the frequency band of interest. Systolic peaks are then located using a peak-detection algorithm (e.g. based on a minimum peak height and a minimum inter-peak distance), and diastolic points are identified as local minima or secondary peaks occurring between consecutive systolic peaks.

Once the systolic peaks are located, the time interval between successive peaks, known as the peak-to-peak interval (analogous to the R-R interval in ECG), is measured. The instantaneous heart rate (in beats per minute) is then computed as:

$$\text{Heart Rate (bpm)} = 60 / (\text{peak-to-peak interval in seconds})$$

This experiment illustrates how digital signal processing techniques translate directly into clinically meaningful measurements extracted from biomedical signals.

Experiment 11: Power Spectral Density of a Square Wave

Objective

- To understand the concept of power spectral density (PSD) as a description of how the power of a signal is distributed over frequency.
- To generate a square wave and compute and plot its power spectral density.

Theory

A square wave is a periodic signal that alternates between two fixed levels and, unlike a pure sinusoid, contains energy at its fundamental frequency as well as at all of its odd harmonics ($3f_0$, $5f_0$, $7f_0$, ...), with harmonic amplitudes decreasing as $1/n$. The power spectral density describes how the average power of a signal is distributed as a function of frequency, and for a periodic signal it consists of a series of discrete lines (a line spectrum) located at the fundamental frequency and its harmonics.

In this experiment, a square wave of a chosen fundamental frequency is generated over a fixed duration and sampling rate. Its power spectral density is then estimated, commonly using the periodogram method, which computes the squared magnitude of the signal's DFT/FFT and normalises it appropriately:

$$PSD(k) = |X(k)|^2 / N$$

MATLAB provides functions such as `periodogram` or `pwelch` for this purpose (Python provides `scipy.signal.periodogram` or `welch`). The resulting PSD plot is expected to show a dominant spectral line at the fundamental frequency and progressively smaller lines at the odd harmonics, confirming the theoretical Fourier-series composition of a square wave.

Experiment 12: Fourier Series Approximation of a Square Wave

Objective

- To understand how a periodic square wave can be represented as an infinite sum of harmonically related sinusoids (its Fourier series).
- To numerically reconstruct a square wave by summing a finite number of Fourier series terms and observe convergence and the Gibbs phenomenon.

Theory

Any periodic signal that satisfies the Dirichlet conditions can be expressed as a sum of sinusoids (or complex exponentials) at the fundamental frequency and its integer multiples (harmonics) – this is the Fourier series representation. For an odd, symmetric square wave of amplitude A and fundamental angular frequency ω_0 , only the odd harmonics are present, and the trigonometric Fourier series is:

$$x(t) \approx (4A/\pi) [\sin(\omega_0 t) + (1/3) \sin(3\omega_0 t) + (1/5) \sin(5\omega_0 t) + (1/7) \sin(7\omega_0 t) + \dots]$$

Each additional term added to this series brings the partial-sum waveform progressively closer to the ideal square wave. In this experiment, the square wave is reconstructed approximately by summing a finite number, N , of these odd-harmonic sine terms in MATLAB/Python, and the resulting approximation is plotted for increasing values of N (e.g. $N = 1, 3, 5, 10, 50$ terms).

As more harmonics are included, the approximation becomes visually closer to a true square wave; however, a persistent overshoot (of approximately 9% of the jump discontinuity) is observed near the sharp edges of the waveform even as N becomes large. This overshoot, which does not vanish but only becomes narrower with increasing N , is known as the Gibbs phenomenon and is a fundamental characteristic of Fourier series approximation of signals containing discontinuities.

Experiment 13: Amplitude Spectrum and Fourier Transform Approximation of a Multi-Frequency Signal

Objective

- To find the amplitude spectrum of a signal composed of multiple sinusoidal frequency components.
- To numerically approximate the continuous Fourier transform integral of the signal over a specified frequency range (0 to 900 Hz).

Theory

A multi-frequency signal is formed by superposing two or more sinusoids of different frequencies and amplitudes, for example $x(t) = A_1 \sin(2\pi f_1 t) + A_2 \sin(2\pi f_2 t) + \dots$. The amplitude spectrum of such a signal shows discrete peaks at each of the constituent frequencies $f_1, f_2, \dots$, with peak heights proportional to the corresponding amplitudes $A_1, A_2, \dots$. In this experiment, the composite signal is first sampled at a sufficiently high rate (satisfying the Nyquist criterion for the highest frequency component present) and its DFT/FFT is computed to obtain the discrete amplitude spectrum, which is plotted against frequency in Hz.

In addition to the DFT-based (discrete) spectrum, the continuous-time Fourier transform integral of the signal, defined as:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt$$

is approximated numerically by evaluating this integral (via numerical summation/quadrature over the sampled signal) at closely spaced frequency points across the range $0 \leq f \leq 900$ Hz. Comparing this numerically approximated continuous spectrum with the discrete FFT-based spectrum illustrates the relationship between the DFT and the underlying continuous Fourier transform, and shows how each spectral peak's location correctly identifies one of the signal's constituent frequencies.

Experiment 14: Discrete-Time Fourier Transform (DTFT) of a Finite-Duration Sequence

Objective

- To determine the Discrete-Time Fourier Transform of a given finite-duration sequence at a specified number of equally spaced frequency points.
- To plot the magnitude, phase (angle), real part and imaginary part of the resulting DTFT.

Theory

The Discrete-Time Fourier Transform (DTFT) expresses a discrete-time sequence $x(n)$ as a continuous function of digital frequency ω (in radians), and is defined by the summation:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n) e^{-j\omega n}$$

For a finite-duration sequence $x(n)$, the sum reduces to a finite number of terms, but the resulting transform $X(e^{j\omega})$ is still a continuous, periodic function of ω with period 2π . In practice, this continuous function is evaluated (sampled) at a large but finite number of equally spaced frequency points – in this experiment, 501 points – over the range $-\pi \leq \omega \leq \pi$, which provides a very close approximation to the true continuous spectrum.

Since $X(e^{j\omega})$ is, in general, a complex-valued function, it is fully characterised by four real quantities computed at each frequency point: the magnitude $|X(e^{j\omega})|$, the phase angle $\angle X(e^{j\omega})$, the real part $\text{Re}\{X(e^{j\omega})\}$, and the imaginary part $\text{Im}\{X(e^{j\omega})\}$. Each of these four quantities is computed and plotted separately against ω in MATLAB/Python, giving a complete picture of the frequency-domain behaviour of the given finite-duration sequence.